

STATUS: PASSED & VERIFIED

Technical Assignment Report

The Untested API — Comprehensive Testing, Bug Resolution & Feature Engineering

PROJECT	CANDIDATE REPOSITORY	TECH STACK	TEST RESULTS
Task Manager API	github.com/SWapnit18/Underpin	Node.js, Express, Jest, Supertest	87 / 87 Passed (100%)

1. Executive Summary

This report documents the testing, debugging, and feature development performed for the **Untested API** take-home assignment. The primary objectives were to design a production-grade automated test suite from scratch, identify and document latent bugs, fix critical regressions, and implement a validated task assignment endpoint (`PATCH /tasks/:id/assign`) with clean REST semantics.

- Overall Line Coverage:** 97.27% (exceeding the 80% requirement).
- Total Test Cases:** 87 automated tests spanning unit and integration layers with 0 failures.
- Bugs Resolved:** Fixed priority data loss on task completion and off-by-one pagination skipping.
- New Feature Delivered:** Added assignee validation, persistence, and HTTP 409 conflict detection for already-assigned tasks.

2. Automated Testing & Code Coverage

Testing was divided into three dedicated suites following clean separation of concerns:

Test Suite	File Path	Tests	Scope & Capabilities
Service Unit Tests	task-api/tests/taskService.test.js	40 tests	CRUD operations, store isolation, pagination limits, stats calculation, and regression safety.
Validation Unit Tests	task-api/tests/validators.test.js	17 tests	Input boundary testing: empty strings, whitespace, type enforcement, invalid statuses, ISO dates.
Routes Integration Tests	task-api/tests/tasks.routes.test.js	30 tests	End-to-end Supertest execution across all HTTP methods, query filters, pagination, and status codes (200, 201, 204, 400, 404, 409).

Coverage Metric Breakdown

Component	Statement %	Branch %	Function %	Line %
src/routes/tasks.js	100.00%	100.00%	100.00%	100.00%
src/services/taskService.js	100.00%	96.00%	100.00%	100.00%
src/utils/validators.js	100.00%	100.00%	100.00%	100.00%
src/app.js	69.23%	75.00%	0.00%	69.23%

Component	Statement %	Branch %	Function %	Line %
Total Project Suite	97.51%	97.87%	93.33%	97.27%

3. Bug Reports & Engineering Fixes

Bug 1: Task Completion Overwrites Existing Priority FIXED

Location: `src/services/taskService.js:69` | **Severity:** High

Root Cause: The `completeTask(id)` function hardcoded `priority: 'medium'` upon task completion. If a user had created an urgent or low-priority task, completing it permanently erased and corrupted the priority value.

Resolution: Removed the hardcoded priority property so the existing task's attributes are fully preserved via object spread (`...task`).

Bug 2: Off-By-One Offset in Pagination Calculation FIXED

Location: `src/services/taskService.js:12` | **Severity:** High

Root Cause: Offset was computed as `offset = page * limit`. When requesting page 1 with limit 2, offset evaluated to `1 * 2 = 2`, skipping items 0 and 1 entirely.

Resolution: Corrected formula to `(pageNum - 1) * limitNum` with defensive numeric parsing (`Math.max(1, parseInt(page) || 1)`).

Bug 3: Loose Status Substring Matching

Location: `src/services/taskService.js:9` | **Severity:** Medium

Root Cause: Status filtering used `t.status.includes(status)` instead of strict equality (`===`), allowing partial terms like `status=do` to match both `todo` and `done`.

Status: Formally documented with a diagnostic test in `tests/taskService.test.js`.

4. New Feature: Task Assignment Endpoint

Implemented endpoint: `PATCH /tasks/:id/assign`

Key Architectural Decisions

- **Input Validation** (`src/utils/validators.js`): Validates that the `assignee` field is present, is a string, and contains non-whitespace characters. Rejects empty payloads with HTTP 400.
- **Conflict Protection** (`src/services/taskService.js`): If a task already has an assignee, returns `{ conflict: true, task }` which the route maps to **HTTP 409 Conflict**. This prevents race conditions and accidental overwrites in multi-user environments.
- **Resource Verification:** If the target ID does not exist, returns **HTTP 404 Not Found**.

5. Production Readiness & Engineering Reflection

1. **Data Persistence:** The current implementation relies on an in-memory array. Before production deployment, an ACID-compliant database (PostgreSQL with Prisma/Knex) should replace in-memory arrays to prevent data loss on

server restarts.

2. **Concurrency & Race Conditions:** In a high-throughput multi-worker setup, concurrent writes to the same task must be protected using database transactions or optimistic concurrency control (e.g. version tokens).
3. **Security & Auth:** Add authentication (JWT / OAuth2), role-based access control (RBAC), Helmet security headers, and Express rate limiting.

6. Verification & Reproducibility Guide

```
# 1. Clone repository git clone https://github.com/Swapnit18/Underpin.git cd Underpin # 2. Run automated test suite npm test # 3. Generate complete coverage report npm run coverage
```

Report generated on October 1, 2026 • Candidate: Swapnil Patel • Repository: github.com/Swapnit18/Underpin